

Comment faire un TIPE en moins de 5Mo ?

Antoine THEOBALD-ROSA

N° SCEI 20978

2024-2025

Plan

- 1 Introduction
- 2 Mon approche
- 3 Implémentation algorithmique
- 4 Performances & Optimisation
- 5 Conclusion
- 6 Annexe

(Fichier PDF uniquement, 5Mo maximum)

Figure: Extrait de la page de remise du TIPE

Problème à résoudre

Différentes options,

Mode de compression	Taux de compression
JPEG (avec perte)	10 à 20
PNG (sans perte)	2 à 3
GIF (sans perte, 256 couleurs)	3 à 5

Problème à résoudre

Différentes options,

Mode de compression	Taux de compression
JPEG (avec perte)	10 à 20
PNG (sans perte)	2 à 3
GIF (sans perte, 256 couleurs)	3 à 5

→ Besoin de mieux

Cheminement vers mon approche

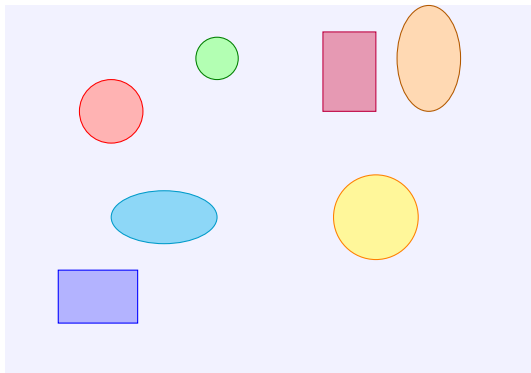


Figure: Exemple d'utilisation de TikZ

Cheminement vers mon approche

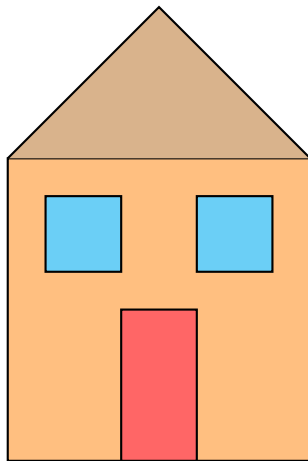
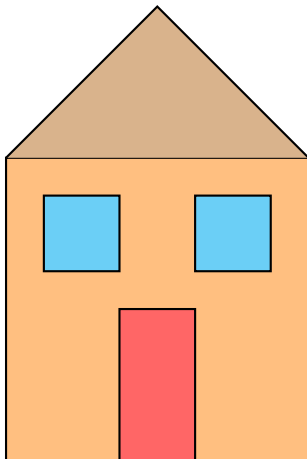


Figure: Dessin fait avec TikZ

Cheminement vers mon approche



```
\begin{tikzpicture}
  % Corps de la maison
  \fill[orange!50] (0,0) rectangle (4,4);
  \draw[thick] (0,0) rectangle (4,4);

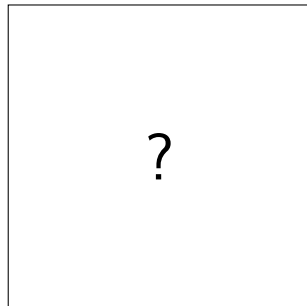
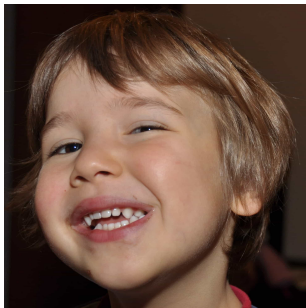
  % Toit
  \fill[brown!60] (0,4) -- (2,6) -- (4,4) -- cycle;
  \draw[thick] (0,4) -- (2,6) -- (4,4);

  % Porte
  \fill[red!60] (1.5,0) rectangle (2.5,2);
  \draw[thick] (1.5,0) rectangle (2.5,2);

  % Fenêtre gauche
  \fill[cyan!50] (0.5,2.5) rectangle (1.5,3.5);
  \draw[thick] (0.5,2.5) rectangle (1.5,3.5);

  % Fenêtre droite
  \fill[cyan!50] (2.5,2.5) rectangle (3.5,3.5);
  \draw[thick] (2.5,2.5) rectangle (3.5,3.5);
\end{tikzpicture}
```


Problème à résoudre



Plan

- 1 Introduction
- 2 **Mon approche**
- 3 Implémentation algorithmique
- 4 Performances & Optimisation
- 5 Conclusion
- 6 Annexe

Idée de l'algorithme :

Initialiser une image vide (canvas)

POUR un certain nombre d'itérations :

 Générer des formes aléatoires (position, couleur, taille...)

 Garder les meilleures formes générées et
 en générer des variantes (mutations)

 Répéter le processus avec les variantes

Dessiner la meilleure forme sur le canvas

Idée de l'algorithme :

Initialiser une image vide (canvas)

POUR un certain nombre d'itérations :

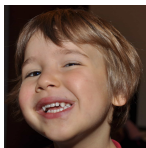
 Générer des formes aléatoires (position, couleur, taille...)

 Garder les **meilleures formes** générées et
 en générer des variantes (mutations)

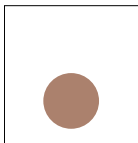
 Répéter le processus avec les variantes

Dessiner la meilleure forme sur le canvas

Calcul de distance



—



Distance de Manhattan	Erreur quadratique totale
$\sum_{i \in \text{pixel}} A[i] - B[i] $ Rapide à calculer Moins précise	$\sqrt{\frac{1}{N} \sum_{i \in \text{pixel}} A[i] - B[i] ^2}$ Lente à calculer Plus précise

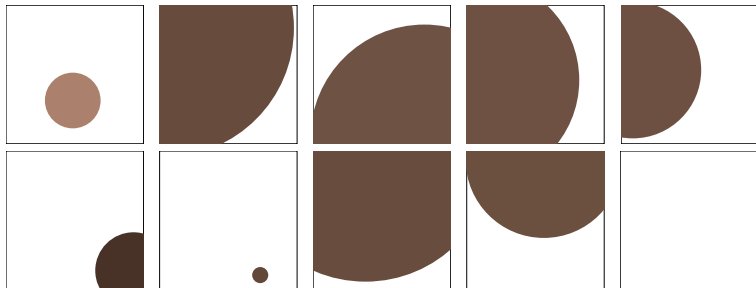
$$\approx 127 \times 10^6$$

$$\approx 126 \times 10^3$$

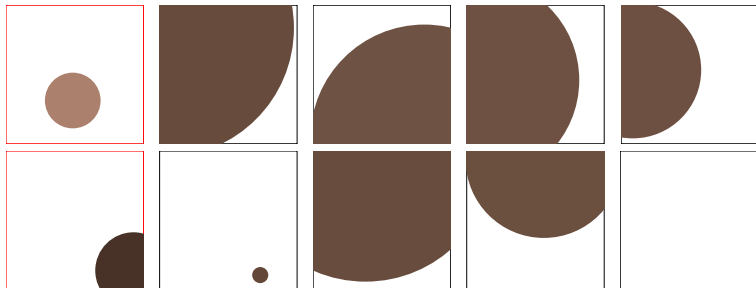
Plan

- 1 Introduction
- 2 Mon approche
- 3 Implémentation algorithmique**
- 4 Performances & Optimisation
- 5 Conclusion
- 6 Annexe

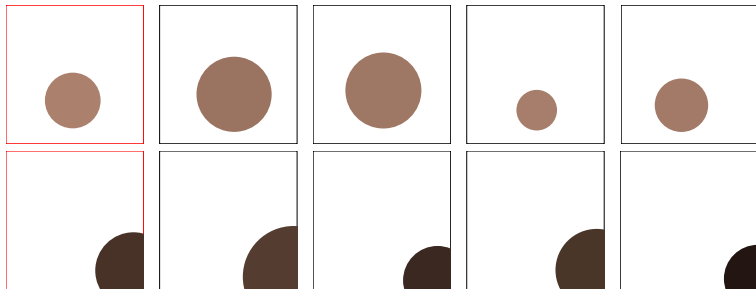
Exécution de l'algorithme



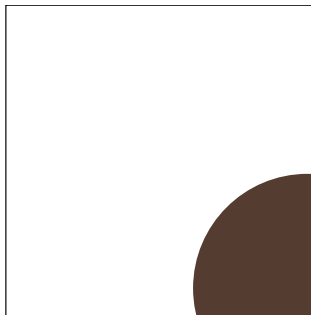
Exécution de l'algorithme



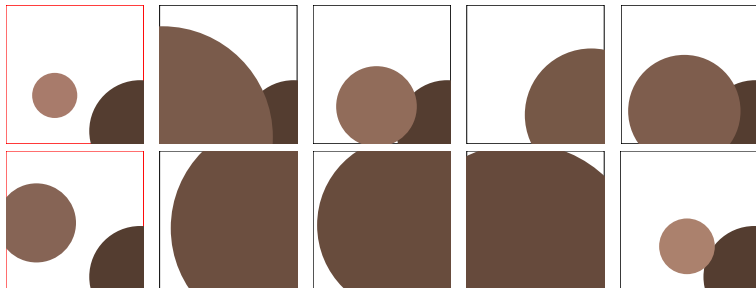
Exécution de l'algorithme



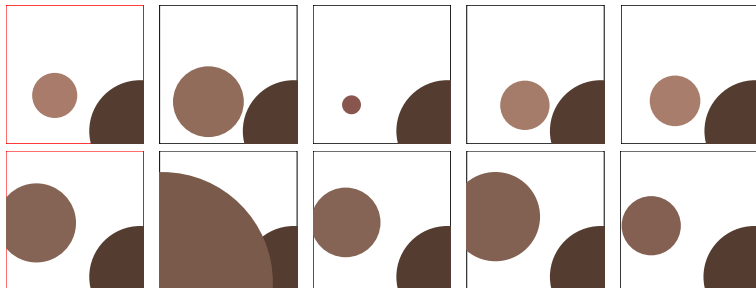
Exécution de l'algorithme



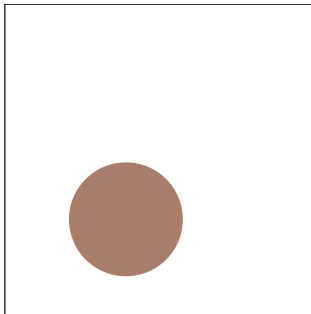
Exécution de l'algorithme



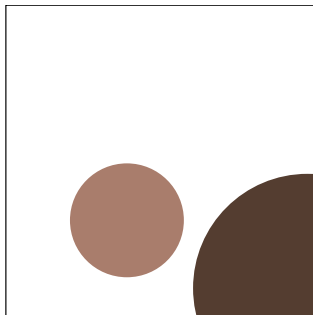
Exécution de l'algorithme



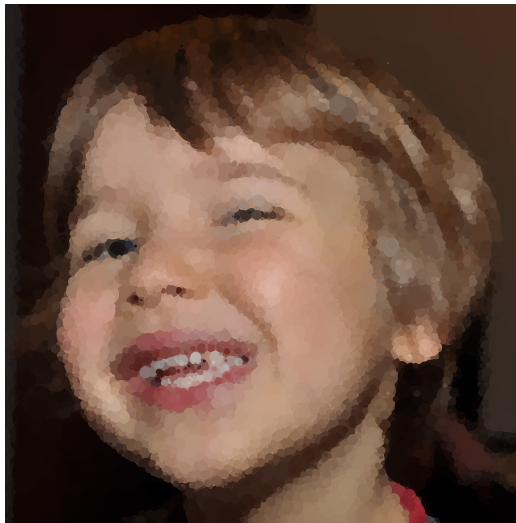
Exécution de l'algorithme



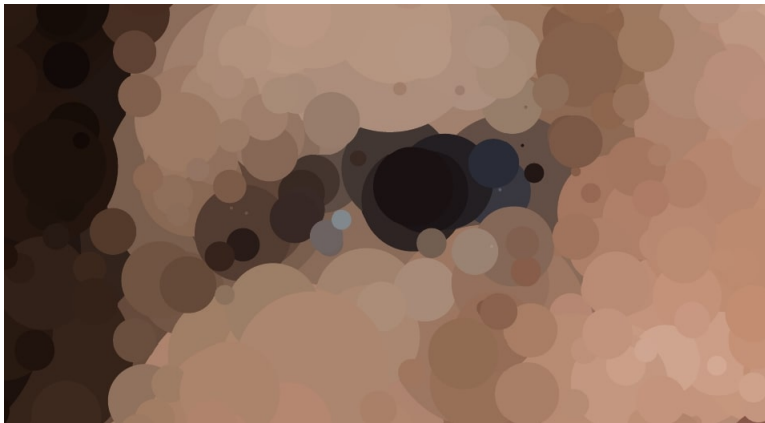
Exécution de l'algorithme



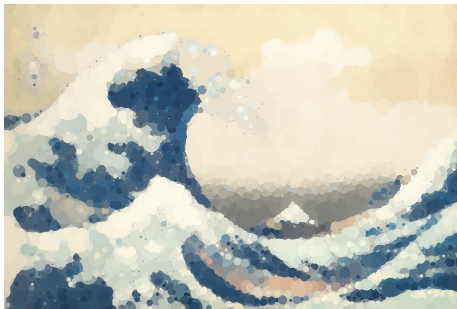
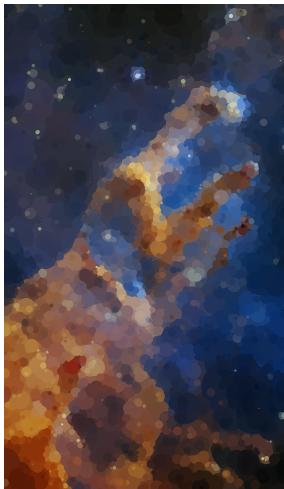
Résultat



Paramètres : 6000 cercles, 160 cercles générés et 20 sélectionnés pour les mutations, 2 générations.

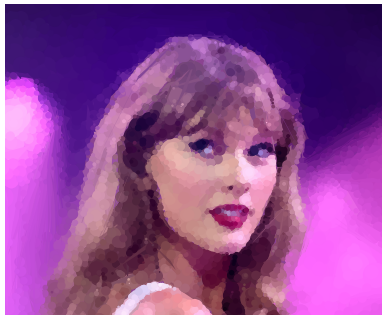
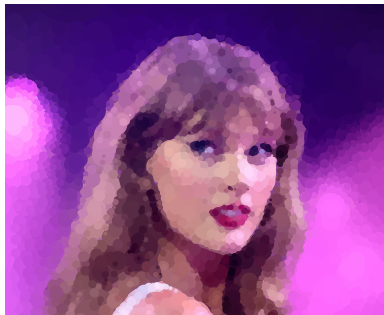


D'autres exemples



A gauche, 2500 cercles. A droite, 3000 cercles.

Avec des triangles



Plan

- 1 Introduction
- 2 Mon approche
- 3 Implémentation algorithmique
- 4 Performances & Optimisation**
- 5 Conclusion
- 6 Annexe

Performance en espace

Image originale (PNG)	Image compressée (SVG)
4,389 MB 4389 kB	0,279 MB 279 kB

Rapport de compression : $\approx 15,3$

```
<circle cx="1005" cy="777" r="1606" fill="rgb(102,74,60)" />  
<circle cx="914" cy="973" r="518" fill="rgb(168,128,107)" />  
<circle cx="437" cy="1964" r="503" fill="rgb(56,36,27)" />
```

Figure: Extrait du fichier svg

Compression du fichier

```
<circle cx="1005" cy="777" r="1606" fill="rgb(102,74,60)" />  
<circle cx="914" cy="973" r="518" fill="rgb(168,128,107)" />  
<circle cx="437" cy="1964" r="503" fill="rgb(56,36,27)" />
```

Figure: Extrait du fichier svg

Compression du fichier

```
<circle cx="1005" cy="777" r="1606" fill="rgb(102,74,60)" />
```

1005 777 1606 102 74 60

1005~~0~~777~~7~~1606~~1~~02~~0~~74~~0~~60

1 ligne de code $\iff$ 21 caractères

Compression du fichier

Caractère	Encodage
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
∅	1010

1 ligne de code $\iff 21 \times 4 = 84$ bits

Compression du fichier

Image originale (PNG)	Image compressée (JPG)	Fichier compressé
4,389 MB 4389 kB	0,473 MB 473 kB	0,063 MB 63 kB

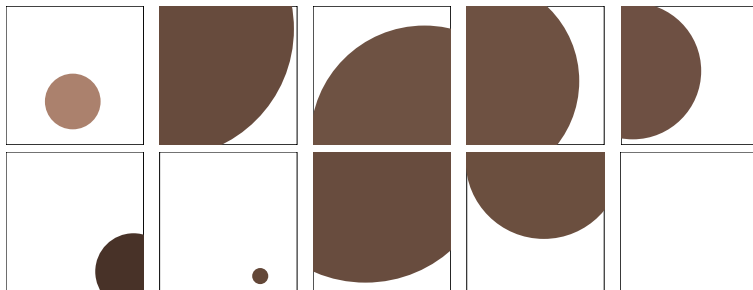
Rapport de compression :

- 69,7 par rapport à PNG
- 7,5 par rapport à JPG

Performance du programme

Temps de générations	RAM utilisé
≈ 64 minutes	≤ 100 MB

Optimisation du temps d'exécution



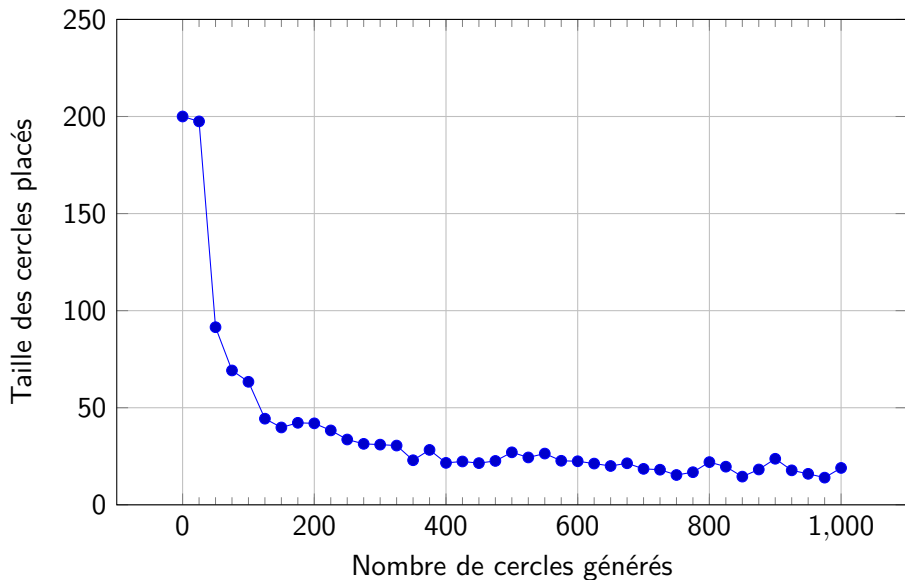
⇒ Parallélisable

Optimisation du temps d'exécution

```
for(int i = Nselected; i < Nit; i++){
    for(int t = 0; t < NUM_THREADS - 1; t++){
        thread_args[t].target_im = target_im;
        thread_args[t].blank = blank;
        thread_args[t].shapes = shapes;
        thread_args[t].scores = scores;
        thread_args[t].i = i;
        i++;
        // Create the thread and pass the structure
        if (pthread_create(&threads[t], NULL, thread_function, &thread_args[t]) != 0) {
            perror("Failed to create thread");
            exit(EXIT_FAILURE);
        }
    }
    // Wait for all threads to finish
    for (int t = 0; t < NUM_THREADS - 1; t++) {
        pthread_join(threads[t], NULL);
    }
}
```

Implémentation avec 8 threads :
≈ 21 minutes

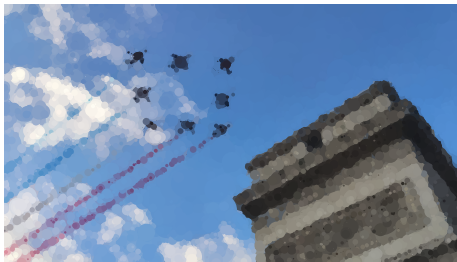
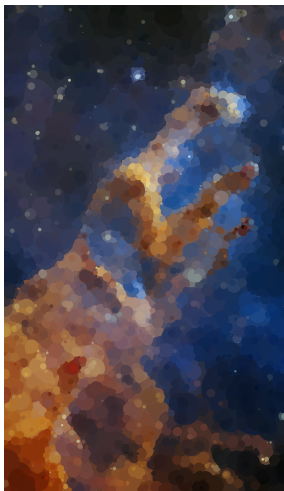
Analyse de l'algorithme



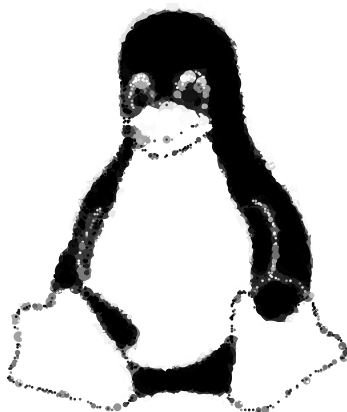
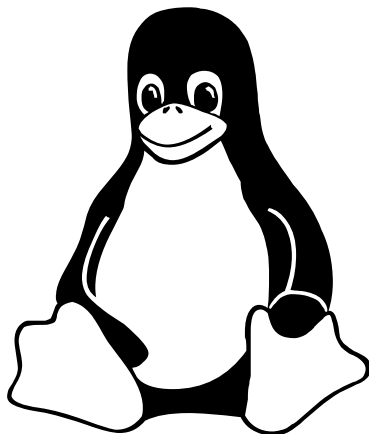
Plan

- 1 Introduction
- 2 Mon approche
- 3 Implémentation algorithmique
- 4 Performances & Optimisation
- 5 Conclusion
- 6 Annexe

Analyse des résultats

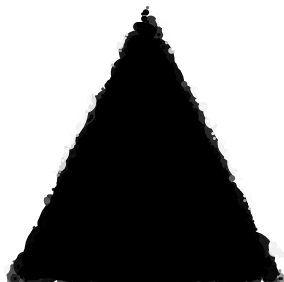
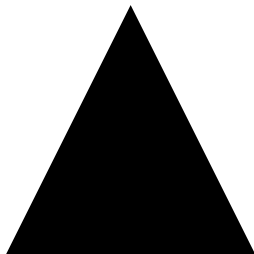


Analyse des résultats



A gauche, fichier SVG de 4 kB. A droite, fichier SVG de 30 kB (compressé)

Analyse des résultats



A gauche, fichier SVG de 0.2 kB. A droite, fichier SVG de 27 kB (compressé)

Avantages et Inconvénients

Inconvénients	Avantages
Temps de génération	

Avantages et Inconvénients

Inconvénients	Avantages
Temps de génération	
Mauvais pour les images détaillées	

Avantages et Inconvénients

Inconvénients	Avantages
Temps de génération	
Mauvais pour les images détaillées	
Mauvais pour les logos simples	

Avantages et Inconvénients

Inconvénients	Avantages
Temps de génération	Compression très efficace
Mauvais pour les images détaillées	
Mauvais pour les logos simples	

Avantages et Inconvénients

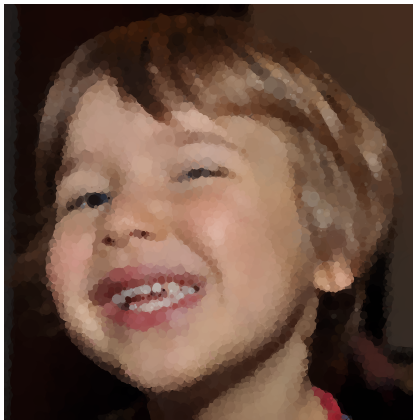
Inconvénients	Avantages
Temps de génération	Compression très efficace
Mauvais pour les images détaillées	Poids de l'image indépendant de sa taille
Mauvais pour les logos simples	

Avantages et Inconvénients

Inconvénients	Avantages
Temps de génération	Compression très efficace
Mauvais pour les images détaillées	Poids de l'image indépendant de sa taille
Mauvais pour les logos simples	Méthode

Problématique

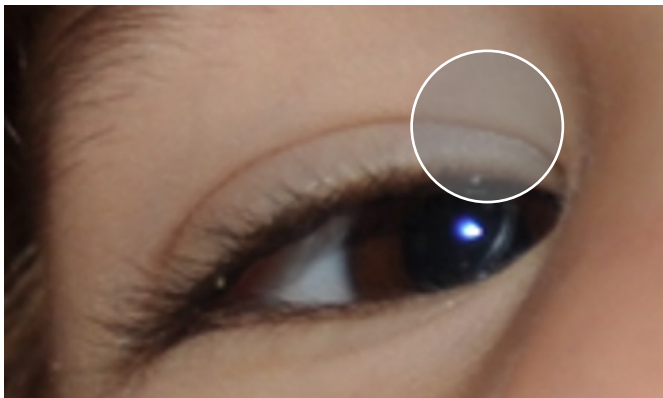
Comment faire un TIPE en moins de 5Mo ?



Plan

- 1 Introduction
- 2 Mon approche
- 3 Implémentation algorithmique
- 4 Performances & Optimisation
- 5 Conclusion
- 6 Annexe

Calcul des couleurs des cercles

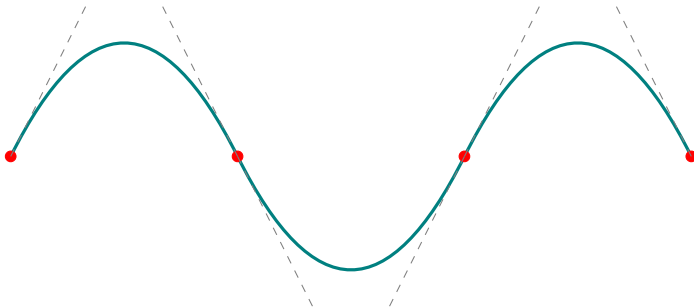


Poids selon le nombre de cercles

Nombre de cercles	Poids du fichier SVG compressé
500 cercles	0,005 MB
1000 cercles	0.01 MB
2000 cercles	0.02 MB
4000 cercles	0.04 MB
6000 cercles	0.06 MB
8000 cercles	0.08 MB

1 MB = 1 Mo = 1,048,576 octets (bytes)

Courbes de Bézier



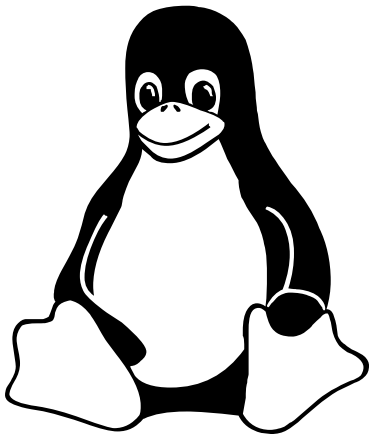


Figure: Logo Linux, poids 4 kB

Thèse : *Compact image vectorization by stochastic approaches*

Amélioration du temps d'exécution

CPU	GPU
8 cœurs	5000 à 10000 cœurs

Paralléliser le calcul de la meilleure forme sur la carte graphique.

Code

```
#include <cairo.h>
#include <cairo-pdf.h>
#include <math.h>
#include <stdbool.h>
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h> // For random seeds
#include <pthread.h>

/*
All the images are expected to be in "img" folder.
*/

// Longest parameters : 250/25/5/5000
// Long : 160/20/5/8000
const int Nit = 160; // number of shapes generated in each iterations
const int Nselected = 20; // number of shapes selected during each iteration; must be a divisor of Nit
const int Ngen = 2; // Number of generations
const int Nshape = 8000;

typedef struct color{
    uint8_t r;
    uint8_t g;
    uint8_t b;
    uint8_t a;
} color;
```

Code

```
typedef struct ImageData {
    int width;
    int height;
    int stride; // number of bytes of a single row in the representation of the image
    color avg_col;
    unsigned char *data; // 1D array representing the pixels of the image
} ImageData;

typedef struct Circle {
    int centerx;
    int centery;
    int radius;
    color c;
} Circle;

typedef struct Triangle{
    int x1; int y1;
    int x2; int y2;
    int x3; int y3;
    color c;
} Triangle;

typedef enum { CIRCLE, TRIANGLE } ShapeType;

typedef struct Shape {
    ShapeType type;
    union {
        Circle circle;
        Triangle triangle;
    };
} Shape;
```


Code

```
#define NUM_THREADS 10
typedef struct {
    ImageData *blank;
    ImageData *target_im;
    uint64_t *scores;
    Shape *shapes;
    int i;
} ThreadArgs;

#define MAX_COMMAND_LENGTH 256

////////////////////////////////////
//////////////////////////////////// Misc //////////////////////////////////////
////////////////////////////////////

int max(int a, int b){
    if (a > b) return a;
    return b;
}

int min(int a, int b){
    if (a < b) return a;
    return b;
}

// Returns a random integer in the range [a, b[
int rd(int a, int b){
    return rand () % (b - a + 1 ) + a;
}
```

Code

```

//////////////////////////////////// Images //////////////////////////////////////
////////////////////////////////////

// Returns the average color of an image represented by its ImageData structure
// Tested on 18/09/24
color get_avg_color_img(ImageData *im){
    uint64_t total_red = 0;
    uint64_t total_green = 0;
    uint64_t total_blue = 0;
    uint64_t total_alpha = 0;

    // Iterate through the image's pixels
    for (int y = 0; y < im->height; y++) {
        for (int x = 0; x < im->width; x++) {
            // This is because im->data is a 1D array representing the pixels of the image
            unsigned char *pixel = im->data + y * im->stride + x * 4; // Each pixel takes 4 bytes (ARGB)
            total_alpha += pixel[3];
            total_red += pixel[2];
            total_green += pixel[1];
            total_blue += pixel[0];
        }
    }

    int nb_pixel = (im->height+1)*(im->width+1);
    color avg;
    avg.a = total_alpha/nb_pixel;
    avg.r = total_red/nb_pixel;
    avg.g = total_green/nb_pixel;
    avg.b = total_blue/nb_pixel;
    return avg;
}

```

Code

```
void update_avg_color(ImageData *im){
    im->avg_col = get_avg_color_img(im);
}

// Function to load an image and return the corresponding ImageData structure
// Tested on 18/09/24
ImageData *load_image(const char *filename) {
    char path[256];
    snprintf(path, sizeof(path), "img/%s", filename);

    cairo_surface_t *surface = cairo_image_surface_create_from_png(path);
    if (cairo_surface_status(surface) != CAIRO_STATUS_SUCCESS) {
        fprintf(stderr, "Failed to load image from %s\n", path);
        cairo_surface_destroy(surface);
        exit(EXIT_FAILURE); // Exit if image loading fails
    }

    // Create ImageData structure and store image info
    ImageData *img_data = malloc(sizeof(ImageData));
    img_data->width = cairo_image_surface_get_width(surface);
    img_data->height = cairo_image_surface_get_height(surface);
    img_data->stride = cairo_image_surface_get_stride(surface);
    img_data->data = cairo_image_surface_get_data(surface);
    img_data->avg_col = get_avg_color_img(img_data);

    return img_data;
}
```

Code

```
// Function to create a blank image with a specified width and height
// Tested on 18/09/24
ImageData *create_blank_image(int width, int height) {
    // Create a Cairo surface to store the image
    cairo_surface_t *surface = cairo_image_surface_create(CAIRO_FORMAT_ARGB32, width, height);
    if (cairo_surface_status(surface) != CAIRO_STATUS_SUCCESS) {
        fprintf(stderr, "Failed to create blank image.\n");
        cairo_surface_destroy(surface);
        exit(EXIT_FAILURE);
    }

    // Get the image data and stride
    unsigned char *data = cairo_image_surface_get_data(surface);
    int stride = cairo_image_surface_get_stride(surface);

    // Initialize the image with a white background (ARGB: A=255, R=255, G=255, B=255)
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            unsigned char *pixel = data + y * stride + x * 4;
            pixel[3] = 255; // Alpha channel (fully opaque)
            pixel[2] = 255; // Red channel
            pixel[1] = 255; // Green channel
            pixel[0] = 255; // Blue channel
        }
    }

    // Mark the surface as dirty so that Cairo knows the data has changed
    cairo_surface_mark_dirty(surface);
}
```

Code

```
// Create the ImageData structure and fill it with the necessary information
ImageData *img_data = malloc(sizeof(ImageData));
img_data->width = width;
img_data->height = height;
img_data->stride = stride;
img_data->data = data;
img_data->avg_col.a = 255;
img_data->avg_col.r = 255;
img_data->avg_col.g = 255;
img_data->avg_col.b = 255;

return img_data;
}

// Function to create a new ImageData by copying an existing one
// Semi-Tested on 24/09/24
// Remains to test that the two images are really distinct in the memory (that we made a deep copy)
ImageData *copy_image(ImageData *src) {
    ImageData *new_img = malloc(sizeof(ImageData));
    new_img->width = src->width;
    new_img->height = src->height;
    new_img->stride = src->stride;
    new_img->avg_col = src->avg_col;

    // Allocate memory for the new image data
    new_img->data = malloc(src->stride * src->height);

    // Copy the original image data into the new image
    memcpy(new_img->data, src->data, src->stride * src->height);

    return new_img;
}
```

Code

```
void free_img(ImageData *img) {
    free(img->data);
    free(img);
}

// Function to save the blank image as a PNG file
// Tested on 18/09/24
void save_image_as_png(ImageData *img, const char *filename) {
    cairo_surface_t *surface = cairo_image_surface_create_for_data(img->data,
                                                                    CAIRO_FORMAT_ARGB32,
                                                                    img->width,
                                                                    img->height,
                                                                    img->stride);

    cairo_surface_write_to_png(surface, filename);
    cairo_surface_destroy(surface);
}
```

Code

```
// Returns the RMS distance between two images
// A score close to 0 means that the images are close to one-another
// Tested on 25/09/24
uint64_t RMS_img(ImageData *im1, ImageData *im2){
    assert(im1->height == im2->height && im1->width == im2->width);
    uint64_t RMS = 0;

    // Iterate through the image's pixels
    for (int y = 0; y < im1->height; y++) {
        for (int x = 0; x < im1->width; x++) {
            // This is because im->data is a 1D array representing the pixels of the image
            unsigned char *pixel1 = im1->data + y * im1->stride + x * 4;
            unsigned char *pixel2 = im2->data + y * im2->stride + x * 4;
            RMS += (pixel1[0]-pixel2[0])*(pixel1[0]-pixel2[0]) + (pixel1[1]-pixel2[1])*(pixel1[1]-pixel2[1]) +
        }
    }

    return RMS;
}
```

Code

```
// Returns the Manhattan distance between two images
// A score close to 0 means that the images are close to one-another
// Tested on 25/09/24
uint64_t ABS_img(ImageData *im1, ImageData *im2){
    assert(im1->height == im2->height && im1->width == im2->width);
    uint64_t RMS = 0;

    // Iterate through the image's pixels
    for (int y = 0; y < im1->height; y++) {
        for (int x = 0; x < im1->width; x++) {
            // This is because im->data is a 1D array representing the pixels of the image
            unsigned char *pixel1 = im1->data + y * im1->stride + x * 4;
            unsigned char *pixel2 = im2->data + y * im2->stride + x * 4;
            RMS += abs(pixel1[0]-pixel2[0]) + abs(pixel1[1]-pixel2[1]) + abs(pixel1[2]-pixel2[2]);
        }
    }

    return RMS;
}
```


Code

```

//////////////////////////////// Shapes generation //////////////////////////////////
// Computes the average color under the circle in the image
color avg_color_under_circle(ImageData *img, Circle circle) {
    uint64_t sum_r = 0;
    uint64_t sum_g = 0;
    uint64_t sum_b = 0;
    uint64_t sum_a = 0;
    int count = 0;

    // Iterate through the bounding box of the circle
    for (int y = circle.centery - circle.radius; y <= circle.centery + circle.radius; y++) {
        for (int x = circle.centerx - circle.radius; x <= circle.centerx + circle.radius; x++) {
            // Check if the pixel is within the bounds of the image
            if (x >= 0 && x < img->width && y >= 0 && y < img->height) {
                unsigned char *pixel = img->data + y * img->stride + x * 4;
                unsigned char blue = pixel[0];
                unsigned char green = pixel[1];
                unsigned char red = pixel[2];
                unsigned char alpha = pixel[3];
                sum_r += red;
                sum_g += green;
                sum_b += blue;
                sum_a += alpha;
                count++;
            }
        }
    }
}

```

Code

```
// If no pixels were found inside the circle, return black (0,0,0,0)
if (count == 0) {
    // printf("Aucun pixel trouvé dans le cercle\n");
    return (color){0, 0, 0, 0};
}

color avg_color;
avg_color.r = sum_r / count;
avg_color.g = sum_g / count;
avg_color.b = sum_b / count;
avg_color.a = sum_a / count;

return avg_color;
}

// Generate a random circle (random size / center)
// Tested on 24/09/24
Circle rd_circle(ImageData *im){
    Circle circ;
    circ.centerx = rd(0, im->width);
    circ.centery = rd(0, im->height);
    circ.radius = rd(0, min(im->width, im->height)); // Radius are "not too big"
    circ.c = avg_color_under_circle(im, circ);
    return circ;
}
```

Code

```
Circle generate_variation_circle(Circle circ, ImageData *target_img){
    Circle circ_new;

    int width = target_img->width;
    int height = target_img->height;
    uint8_t std_center = 25; // 255 means a lot of variation, 0 means no variation (which makes the function poi
    uint8_t std_radius = 25; // 255 means a lot of variation, 0 means no variation (which makes the function poi
    int dx = rd(0, width*std_center/255);
    int dy = rd(0, height*std_center/255);
    int dr = rd(0, (height+width)*std_radius/(2*255));
    if (rd(0,2) == 1) dx = -dx;
    if (rd(0,2) == 1) dy = -dy;
    if (rd(0,2) == 1) dr = -dr;
    if (circ.centerx + dx < 0) circ_new.centerx = 0;
    else if (circ.centerx + dx > width) circ_new.centerx = width;
    else circ_new.centerx = circ.centerx + dx;
    if (circ.centery + dy < 0) circ_new.centery = 0;
    else if (circ.centery + dy > height) circ_new.centery = height;
    else circ_new.centery = circ.centery + dy;
    if (circ.radius + dr <= 0) circ_new.radius = 1;
    else circ_new.radius = circ.radius + dr; // Radius are "not too big"

    circ_new.c = avg_color_under_circle(target_img, circ_new);
    return circ_new;
}
```

Code

```
// Prints the parametr of a circle (useful to debug)
// Tested on 24/09/24
void printf_Circle(Circle c){
    printf("Parameters of the circle :\n");
    printf("Radius : %d\n", c.radius);
    printf("Center : x = %d, y = %d\n", c.centerx, c.centery);
    printf("Color : RGBA = (%u, %u, %u, %u)\n", c.c.r, c.c.g, c.c.b, c.c.a);
}

void printf_Shape(Shape shape){
    char shape_command[MAX_COMMAND_LENGTH];

    if (shape.type == CIRCLE) {
        printf("Parameters of the circle :\n");
        printf("Radius : %d\n", shape.circle.radius);
        printf("Center : x = %d, y = %d\n", shape.circle.centerx, shape.circle.centery);
        printf("Color : RGBA = (%u, %u, %u, %u)\n", shape.circle.c.r, shape.circle.c.g, shape.circle.c.b, shape.circle.c.a);
    } else {
        printf("Cannot print this shape.");
    }
}
```

Code

// Computes (an approximation of) the average color under a triangle in the image

```
color avg_color_under_triangle(ImageData *img, Triangle triangle) {
    uint64_t sum_r = 0;
    uint64_t sum_g = 0;
    uint64_t sum_b = 0;
    uint64_t sum_a = 0;
    int count = 0;

    int xmin = min(min(triangle.x1, triangle.x2), triangle.x3);
    int ymin = min(min(triangle.y1, triangle.y2), triangle.y3);
    int xmax = max(max(triangle.x1, triangle.x2), triangle.x3);
    int ymax = max(max(triangle.y1, triangle.y2), triangle.y3);

    // Iterate through the bounding box of the circle
    for (int y = ymin; y <= ymax; y++) {
        for (int x = xmin; x <= xmax; x++) {
            // Check if the pixel is within the bounds of the image
            if (x >= 0 && x < img->width && y >= 0 && y < img->height) {
                unsigned char *pixel = img->data + y * img->stride + x * 4;
                unsigned char blue = pixel[0];
                unsigned char green = pixel[1];
                unsigned char red = pixel[2];
                unsigned char alpha = pixel[3];
                sum_r += red;
                sum_g += green;
                sum_b += blue;
                sum_a += alpha;
                count++;
            }
        }
    }
}
```

Code

```
// If no pixels were found inside the circle, return black (0,0,0,0)
if (count == 0) {
    // printf("Aucun pixel trouvé dans le cercle\n");
    return (color){0, 0, 0, 0};
}

color avg_color;
avg_color.r = sum_r / count;
avg_color.g = sum_g / count;
avg_color.b = sum_b / count;
avg_color.a = sum_a / count;

return avg_color;
}

// Generate a random triangle (random size / center)
Triangle rd_triangle(ImageData *im){
    Triangle triangle;
    triangle.x1 = rd(0, im->width);
    triangle.x2 = rd(0, im->width);
    triangle.x3 = rd(0, im->width);
    triangle.y1 = rd(0, im->height);
    triangle.y2 = rd(0, im->height);
    triangle.y3 = rd(0, im->height);
    triangle.c = avg_color_under_triangle(im, triangle);
    return triangle;
}
```

Code

```
Triangle generate_variation_triangle(Triangle triangle, ImageData *target_img){
    Triangle triangle_new;

    int width = target_img->width; int height = target_img->height;
    uint8_t std_center = 25; // 255 means a lot of variation, 0 means no variation
    uint8_t std_radius = 25; // 255 means a lot of variation, 0 means no variation
    int dx1 = rd(0, width*std_center/255); int dy1 = rd(0, height*std_center/255);
    int dx2 = rd(0, width*std_center/255); int dy2 = rd(0, height*std_center/255);
    int dx3 = rd(0, width*std_center/255); int dy3 = rd(0, height*std_center/255);
    if (rd(0,2) == 1) dx1 = -dx1; if (rd(0,2) == 1) dx2 = -dx2;
    if (rd(0,2) == 1) dx3 = -dx3; if (rd(0,2) == 1) dy1 = -dy1;
    if (rd(0,2) == 1) dy2 = -dy2; if (rd(0,2) == 1) dy3 = -dy3;
    if (triangle.x1 + dx1 < 0) triangle.x1 = 0;
    else if (triangle.x1 + dx1 > width) triangle.x1 = width;
    if (triangle.x2 + dx2 < 0) triangle.x2 = 0;
    else if (triangle.x2 + dx2 > width) triangle.x2 = width;
    if (triangle.x3 + dx3 < 0) triangle.x3 = 0;
    else if (triangle.x3 + dx3 > width) triangle.x3 = width;
    if (triangle.y1 + dy1 < 0) triangle.y1 = 0;
    else if (triangle.y1 + dy1 > height) triangle.y1 = height;
    if (triangle.y2 + dy2 < 0) triangle.y2 = 0;
    else if (triangle.y2 + dy2 > height) triangle.y2 = height;
    if (triangle.y3 + dy3 < 0) triangle.y3 = 0;
    else if (triangle.y3 + dy3 > height) triangle.y3 = height;

    triangle_new.x1 = triangle.x1 + dx1; triangle_new.x2 = triangle.x2 + dx2;
    triangle_new.x3 = triangle.x3 + dx3; triangle_new.y1 = triangle.y1 + dy1;
    triangle_new.y2 = triangle.y2 + dy2; triangle_new.y3 = triangle.y3 + dy3;
    triangle_new.c = avg_color_under_triangle(target_img, triangle_new);
    return triangle_new;
}
```

```
// Prints the parameters of a triangle (useful to debug)
void printf_Triangle(Triangle t){
    printf("Parameters of the triangle :\n");
    printf("x1 = %d, y1 = %d\n", t.x1, t.y1);
    printf("x2 = %d, y2 = %d\n", t.x2, t.y2);
    printf("x3 = %d, y3 = %d\n", t.x2, t.y3);
    printf("Color : RGBA = (%u, %u, %u, %u)\n", t.c.r, t.c.g, t.c.b, t.c.a);
}

// Function to draw the provided circle directly on the provided image
// void draw_circle_on_image_en_place(ImageData *img, Shape shape)
void draw_shape_on_image_en_place(ImageData *img, Shape shape) {
    cairo_surface_t *surface = cairo_image_surface_create_for_data(img->data,
                                                                    CAIRO_FORMAT_ARGB32,
                                                                    img->width,
                                                                    img->height,
                                                                    img->stride);

    cairo_t *cr = cairo_create(surface);
```


Code

```
// Set the drawing color
if (shape.type == CIRCLE) {
    cairo_set_source_rgba(cr, (double)shape.circle.c.r/255, (double)shape.circle.c.g/255, (double)shape.circle.c.b/255, (double)shape.circle.c.a/255);
    cairo_arc(cr, shape.circle.centerx, shape.circle.centery, shape.circle.radius, 0, 2 * M_PI);
} else if (shape.type == TRIANGLE) { // TRIANGLE
    cairo_set_source_rgba(cr, (double)shape.triangle.c.r/255, (double)shape.triangle.c.g/255, (double)shape.triangle.c.b/255, (double)shape.triangle.c.a/255);
    cairo_move_to(cr, shape.triangle.x1, shape.triangle.y1);
    cairo_line_to(cr, shape.triangle.x2, shape.triangle.y2);
    cairo_line_to(cr, shape.triangle.x3, shape.triangle.y3);
    cairo_close_path(cr);
} else {
    // Default case (e.g., print an error message or handle the unknown shape type)
    fprintf(stderr, "Error: Unknown shape type.\n");
    cairo_destroy(cr);
    cairo_surface_destroy(surface);
    return;
}

cairo_fill(cr);
cairo_destroy(cr);
cairo_surface_destroy(surface);
cairo_surface_mark_dirty(surface);
}
```

Code

```
// Sort the array of Circle shapes (supposed of length n) according to their score contained in the array scores  
// At all time, scores[i] contains the score of the shape at shapes[i]. Both arrays are modified en-place by the  
// As n will be kept relatively small (n <= 100), we use bubble sort  
void sort_im_score(Shape *shapes, uint64_t *scores, int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (scores[j] > scores[j + 1]) {  
                // Swap scores  
                uint64_t temp_score = scores[j];  
                scores[j] = scores[j + 1];  
                scores[j + 1] = temp_score;  
  
                // Swap shapes  
                Shape temp_shape = shapes[j];  
                shapes[j] = shapes[j + 1];  
                shapes[j + 1] = temp_shape;  
            }  
        }  
    }  
}
```

Code

```
////////////////////////////////////  
//////////////////////////////////// Main //////////////////////////////////////  
////////////////////////////////////  
  
// Write in a file  
void write_txt(const char* filename, const char* shape_command) {  
    FILE *file = fopen(filename, "a");  
    if (file == NULL) {  
        perror("Error opening file");  
        return;  
    }  
    fprintf(file, "%s\n", shape_command);  
    fclose(file);  
}  
  
// Clear the content of a .txt file  
void clear_file(const char* filename) {  
    FILE *file = fopen(filename, "w");  
    if (file != NULL) {  
        fclose(file);  
    } else {  
        perror("Error clearing file");  
    }  
}
```

Code

```
// Given a shape, write in the appropriate file (latex.txt) the latex code corresponding to the shape
void write_shape_latex(int box_width, int box_height, Shape shape) {
    char shape_command[MAX_COMMAND_LENGTH];

    if (shape.type == CIRCLE) {
        // Access the circle member and construct the LaTeX command
        float r = shape.circle.c.r / 255.0;
        float g = shape.circle.c.g / 255.0;
        float b = shape.circle.c.b / 255.0;
        int transformed_y = box_height - shape.circle.centery;

        snprintf(shape_command, MAX_COMMAND_LENGTH,
            "\\filldraw[fill={rgb,1:red,%f;green,%f;blue,%f}, draw=none] (%d,%d) circle (%d);",
            r, g, b, shape.circle.centerx, transformed_y, shape.circle.radius);

        // Write to file
        write_txt("latex.txt", shape_command);

        snprintf(shape_command, MAX_COMMAND_LENGTH,
            "<circle cx=\"%d\\\" cy=\"%d\\\" r=\"%d\\\" fill=\"rgb(%\" PRIu8 \", \"% PRIu8 \", \"% PRIu8 \")\\\" />",
            shape.circle.centerx, transformed_y, shape.circle.radius, shape.circle.c.r, shape.circle.c.g, shap

        // Write to file
        write_txt("svg_file.txt", shape_command);
    }
}
```

Code

```
} else if (shape.type == TRIANGLE) {  
    // Access the triangle member and construct the LaTeX command  
    float r = shape.triangle.c.r / 255.0;  
    float g = shape.triangle.c.g / 255.0;  
    float b = shape.triangle.c.b / 255.0;  
    int transformed_y1 = box_height - shape.triangle.y1;  
    int transformed_y2 = box_height - shape.triangle.y2;  
    int transformed_y3 = box_height - shape.triangle.y3;  
  
    snprintf(shape_command, MAX_COMMAND_LENGTH,  
             "\\fill[fill={rgb,1:red,%f;green,%f;blue,%f}, draw=none] (%d,%d) -- (%d,%d) -- (%d,%d) -- cycle",  
             r, g, b, shape.triangle.x1, transformed_y1, shape.triangle.x2, transformed_y2, shape.triangle.x3, transformed_y3);  
  
    // Write to file  
    write_txt("latex.txt", shape_command);  
  
    snprintf(shape_command, MAX_COMMAND_LENGTH,  
             "<polygon points=\"%d,%d %d,%d %d,%d\" fill=\"rgb(%\" PRIu8 \"%\" PRIu8 \"%\" PRIu8 \"\")\" />",  
             shape.triangle.x1, transformed_y1, shape.triangle.x2, transformed_y2, shape.triangle.x3, transformed_y3);  
  
    // Write the SVG command to the file  
    write_txt("svg_file.txt", shape_command);  
}  
}
```

Code

```
void* thread_function_first_gen(void* arg) {  
    // Unpack the arguments  
    ThreadArgs* data = (ThreadArgs*)arg;  
    ImageData *copy = copy_image(data->blank);  
  
    // >= 0 always generate a circle  
    if (rd(0, 2) >= 0) { // Generate a Circle  
        data->shapes[data->i].type = CIRCLE;  
        data->shapes[data->i].circle = rd_circle(data->target_im);  
    } else { // Generate a Triangle  
        data->shapes[data->i].type = TRIANGLE;  
        data->shapes[data->i].triangle = rd_triangle(data->target_im);  
    }  
  
    draw_shape_on_image_en_place(copy, data->shapes[data->i]);  
    data->scores[data->i] = ABS_img(data->target_im, copy);  
    free_img(copy);  
  
    return NULL;  
}
```

Code

```
void* thread_function(void* arg) {
    // Unpack the arguments
    ThreadArgs* data = (ThreadArgs*)arg;

    ImageData *copy = copy_image(data->blank);
    int index = data->i;

    if (data->shapes[index % Nselected].type == CIRCLE) {
        data->shapes[index].type = CIRCLE;
        data->shapes[index].circle = generate_variation_circle(data->shapes[index % Nselected].circle, data->ta
    }
    else if (data->shapes[index % Nselected].type == TRIANGLE){
        data->shapes[index].type = TRIANGLE;
        data->shapes[index].triangle = generate_variation_triangle(data->shapes[index % Nselected].triangle, da
    }

    draw_shape_on_image_en_place(copy, data->shapes[index]);
    data->scores[index] = ABS_img(data->target_im, copy);
    free_img(copy);
    // printf("image %d processed by thread %d\n", index);

    return NULL;
}
```

Code - Fonction Principale

```
int main() {
    srand(time(0)); // Random seed
    struct timespec start_time; // To measure the time taken by the program. Works with multithreading
    clock_gettime(CLOCK_MONOTONIC, &start_time);

    assert(Nit >= Nselected && Nit % Nselected == 0);
    assert(Nit % NUM_THREADS == 0); // otherwise some threads have more work than others
    // Super important otherwise the program will most likely seg. fault

    char *im_name = "space.png";
    ImageData *target_im = load_image(im_name);
    int width = target_im->width;
    int height = target_im->height;
    ImageData *blank = create_blank_image(width, height);
    uint64_t ABS = ABS_img(target_im, blank);
    // printf("ABS at first : %PRIu64\n", ABS);

    // Clear the .txt and write SVG's header
    clear_file("latex.txt");
    clear_file("svg_file.txt");
    char svg_code[MAX_COMMAND_LENGTH];
    snprintf(svg_code, MAX_COMMAND_LENGTH,
             "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"%d\" height=\"%d\" viewBox=\"0 0 %d %d\">\n"
             width, height, width, height, height);
    write_txt("svg_file.txt", svg_code);

    // Multithreading part
    pthread_t threads[NUM_THREADS];
    ThreadArgs thread_args[NUM_THREADS];
```


Code - Fonction Principale

```
Shape *shapes = malloc(sizeof(Shape) * Nit); // array containing the shapes
uint64_t *scores = malloc(sizeof(uint64_t) * Nit);
char saving_im_name[50]; // Buffer to store the result string

for(int nshape = 0; nshape < Nshape; nshape++){
    printf("On va placer la %d-ème forme.\n", nshape);
    // First iteration
    for(int i = 0; i < Nit; i++){
        for(int t = 0; t < NUM_THREADS - 1; t++){ // t is the id of the thread
            thread_args[t].target_im = target_im;
            thread_args[t].blank = blank;
            thread_args[t].shapes = shapes;
            thread_args[t].scores = scores;
            thread_args[t].i = i;
            i++;

            // Create the thread and pass the structure
            if (pthread_create(&threads[t], NULL, thread_function_first_gen, &thread_args[t]) != 0) {
                perror("Failed to create thread");
                exit(EXIT_FAILURE);
            }
        }

        // Wait for all threads to finish
        for (int t = 0; t < NUM_THREADS - 1; t++) {
            pthread_join(threads[t], NULL);
        }
    }
    sort_im_score(shapes, scores, Nit);
}
```

Code - Fonction Principale

```
// printf("Meilleur score après 1 itération : %"PRIu64"\n", scores[0]);

// Generating evolution
int n_var = Nit/Nselected - 1;
for (int gen = 0; gen < Ngen - 1; gen++){
    for(int i = Nselected; i < Nit; i++){
        for(int t = 0; t < NUM_THREADS - 1; t++){
            thread_args[t].target_im = target_im;
            thread_args[t].blank = blank;
            thread_args[t].shapes = shapes;
            thread_args[t].scores = scores;
            thread_args[t].i = i;
            i++;

            // Create the thread and pass the structure
            if (pthread_create(&threads[t], NULL, thread_function, &thread_args[t]) != 0) {
                perror("Failed to create thread");
                exit(EXIT_FAILURE);
            }
        }

        // Wait for all threads to finish
        for (int t = 0; t < NUM_THREADS - 1; t++) {
            pthread_join(threads[t], NULL);
        }
    }
    sort_im_score(shapes, scores, Nit);
}
```

Code - Fonction Principale

```
// printf("\nMeilleur cercle trouvé pour la %d-ème forme dessinée :\n", nshape);
// printf_Circle(shapes[0]);
// printf("\n");

if(shapes[0].circle.radius > 0){
    write_shape_latex(target_im->width, target_im->height, shapes[0]);
    draw_shape_on_image_en_place(blank, shapes[0]);
} else {
    printf("flop ! rayon négatif");
    nshape--;
}

if ((nshape + 1) % 100 == 0){
    // Create the string with x and "iterations"
    sprintf(saving_im_name, "%d_shapes.png", nshape + 1);
    printf("Saved !\n");
    save_image_as_png(blank, saving_im_name);
}

save_image_as_png(blank, "final.png");

free(shapes);
free(scores);
```

Code - Fonction Principale

```
// Write SVG's footer
snprintf(svg_code, MAX_COMMAND_LENGTH,
         "</g>\n</svg>");
write_txt("svg_file.txt", svg_code);

// End timing
struct timespec end_time;
clock_gettime(CLOCK_MONOTONIC, &end_time);
double total_time = (end_time.tv_sec - start_time.tv_sec) +
                    (end_time.tv_nsec - start_time.tv_nsec) / 1e9;
printf("Execution time: %f seconds\n", total_time);

return 0;
}
```